MODULE M7 · DÉVELOPPEMENT INTERNET · EST DIVTEC, TEEINF2

Clair Obscur

EXPEDITION 33

Documentation technique

Colin Reist · 5 novembre 2026

1. Thème et contenu

Le site est une vitrine non officielle de *Clair Obscur : Expedition 33*, le RPG au tour par tour de Sandfall Interactive sorti en 2025. Il n'a aucun lien avec Sandfall Interactive ni Kepler Interactive : le jeu et ses visuels appartiennent à leurs ayants droit.

Dans le jeu, une entité appelée la Peintresse peint chaque année un nombre sur son monolithe, et tous les habitants de cet âge disparaissent. L'expédition 33 part la combattre avant qu'elle ne peigne le nombre suivant.

Le site présente le jeu sans spoiler et reprend sa direction artistique : fond sombre, accents dorés, typographie à empattements et sections claires « parchemin » qui évoquent le clair-obscur.

PAGE	ROUTE	CONTENU
Accueil	`/`	Accroche, monolithe animé « 33 », pétales qui tombent, tuiles vers les 4 sections
Univers	`/univers`	La ville de Lumière, la Peintresse, le Gommage, les expéditions et quatre figures du continent : Renoir, Esquie, les Gestrals, les Nevrons
Personnages	`/personnages`	Sélecteur interactif des six membres de l'expédition : biographie, mécanique de combat, arme, voix anglaise
Combat	`/combat`	Tour par tour, esquive et parade en temps réel, visée libre, Pictos et Luminas
Coulisses	`/coulisses`	Fiche technique, studio, palmarès des Game Awards 2025, sources

2. Installation et utilisation

Le site se lance en local avec deux commandes, après avoir installé Node.js 24.

Une version en ligne est aussi disponible, sans installation : est-2026-web-dev.vercel.app.

Prérequis

- Node.js 24, fourni avec npm 11 : cette version est nécessaire pour lire le `package-lock.json`
- Un navigateur récent

Lancer le site

```
npm install
npm run dev
```

Le site est alors accessible sur `http://localhost:3000`.

COMMANDE	RÔLE
`npm run dev`	Serveur de développement avec rechargement automatique
`npm run build`	Version de production, pages prérendues en HTML statique
`npm run preview`	Sert la version construite en local
`npm run lint`	Vérifie le style et la qualité du code (ESLint)
`npm run typecheck`	Vérifie les types TypeScript

Navigation

Le site se lit dans cet ordre, puis revient à l'accueil :

1. Accueil
2. Univers
3. Personnages
4. Combat
5. Coulisses

- **Logo** « Expédition 33 », en haut à gauche : retour à l'accueil depuis n'importe quelle page.
- **Menu principal** : Univers, Personnages, Combat, Coulisses. La page active est soulignée en or. Sous 768 px de large, le menu s'ouvre avec le bouton burger.
- **Fil d'Ariane** en tête de chaque page de contenu.
- **Précédent / Suivant** en bas de chaque page, selon l'ordre ci-dessus.
- **Page Personnages** : un clic sur un nom, ou sur les flèches, change le personnage affiché. L'adresse se met à jour (`/personnages#maelle`), ce qui permet de partager un lien vers un personnage précis.
- **Lien « Aller au contenu »**, visible à la tabulation, pour la navigation au clavier.

3. Structure du projet

Le code du site vit dans `app/`, les images dans `public/img/`. Nuxt crée une route par fichier de `app/pages/` et importe automatiquement les composants de `app/components/`.

FICHIER OU DOSSIER	RÔLE
`app/app.vue`	Coquille du site : en-tête, contenu, pied de page, titre, métadonnées et favicon
`app/app.config.ts`	Configuration du thème Nuxt UI : couleurs, conteneur, boutons
`app/assets/css/main.css`	Thème : polices, palette, variables, animations
`app/components/AppHeader.vue`	En-tête et menu, desktop et mobile
`app/components/AppFooter.vue`	Navigation précédent / suivant et mentions
`app/components/AppLogo.vue`	Logo texte « Expédition 33 »
`app/components/PageIntro.vue`	En-tête commun des pages de contenu
`app/components/ArtworkPlaceholder.vue`	Image avec crédit, ou cadre de substitution
`app/components/PetalsDecor.vue`	Pétales animés en SVG sur l'accueil
`app/data/personnages.ts`	Données des six personnages
`app/pages/`	Une page Vue par route : `index`, `univers`, `personnages`, `combat`, `coulisses`
`app/plugins/reveal.ts`	Directive `v-reveal` : apparition au défilement
`app/utils/navigation.ts`	Liste et ordre des pages, utilisés par le menu et le pied de page
`public/favicon.svg`	Favicon vectoriel, utilisé par les navigateurs récents
`public/favicon.ico`	Favicon de secours, en 16, 32 et 48 px
`public/img/`	Visuels des personnages et des figures du continent
`.github/workflows/ci.yml`	Intégration continue : installation, lint et typecheck à chaque push

4. Adaptations du template

Le projet part du Nuxt UI Starter Template (`nuxt-ui-templates/starter`), qui ne fournit qu'une page d'exemple. Tout le contenu et l'identité visuelle ont été créés pour ce projet, sans modifier le code des bibliothèques.

THÈME ET CHARTE GRAPHIQUE (`MAIN.CSS`)

- **Palette « or »**, de 50 à 950, déclarée dans `@theme` (Tailwind CSS v4) et utilisée comme couleur primaire de Nuxt UI.
- **Tokens de couleur** propres au projet (`ground` , `surface` , `ivory` , `sand` , `ash` , `crimson` , `parchment` , `ink` ...), utilisables directement en classes Tailwind (`bg-ground` , `text-ivory`).
- **Variables Nuxt UI rebranchées** (`--ui-bg` , `--ui-text` , `--ui-border` ...) sur ces tokens : tous les composants Nuxt UI adoptent la charte sans être réécrits.
- **Coins droits** (`--ui-radius: 0`) au lieu des coins arrondis par défaut, plus proches de l'esthétique Belle Époque.
- **Classe `.parchment`** : redéfinit localement les variables pour créer les sections claires, fond parchemin et texte encre.
- **Thème sombre unique** : la bascule clair / sombre du template est désactivée (`colorMode: false`).

CONFIGURATION (`NUXT.CONFIG.TS` , `APP.CONFIG.TS`)

- Polices Cormorant Garamond (titres et texte) et Jost (interface), chargées par `@nuxtjs/google-fonts` . Les polices par défaut de Nuxt UI sont désactivées (`fonts: false`).
- Transition de page en fondu (`pageTransition`), stylée dans `main.css` .
- Prérendu de toutes les routes (`routeRules`) : le site est généré en HTML statique.
- `app.config.ts` : couleur primaire `gold` , neutre `stone` , largeur de conteneur et gouttières adaptées, boutons sans arrondi avec la police d'interface.
- Règles ESLint stylistiques ajustées : pas de virgule finale, accolades sur la même ligne.

FAVICON

Le favicon du starter Nuxt est remplacé par un monolithe doré fissuré sur fond sombre, aux couleurs du site (`#C8A25A` sur `#0D0C0B`). Il évoque le monolithe sur lequel la Peintresse peint son nombre.

- **`favicon.svg`** : version vectorielle, nette à toutes les tailles, utilisée par les navigateurs récents.
- **`favicon.ico`** : version de secours en 16, 32 et 48 px, pour les navigateurs qui ne lisent pas le SVG.
- Les deux sont déclarés dans `app.vue` (`useHead`), l'ICO avec l'attribut `sizes` et le SVG avec `type: 'image/svg+xml'` .

COMPOSANTS NUXT UI PERSONNALISÉS

Les composants du template (`UHeader` , `UFooter` , `UPageHero` , `UBreadcrumb` , `UPageCard` , `UAlert` , `UButton` ...) sont conservés mais restylés via leur prop `:ui` .

- `AppHeader` : menu desktop avec soulignement doré de la page active, menu mobile plein écran en grande typographie italique.
- `AppFooter` : remplace le pied de page du starter par une navigation précédent / suivant calculée à partir de `utils/navigation.ts`, et ajoute les mentions légales.

COMPOSANTS CRÉÉS

- `PageIntro` : en-tête commun (fil d'Ariane, titre, introduction), en deux mises en page (`stack` ou `split`), sur un fond en dégradé radial.
- `ArtworkPlaceholder` : affiche une image avec sa légende de crédit, ou un cadre en pointillés quand aucune image n'est fournie.
- `PetalsDecor` : huit pétales SVG qui tombent en boucle sur l'accueil, avec des durées et des délais différents.

JAVASCRIPT AJOUTÉ

- **Directive `v-reveal`** (`plugins/reveal.ts`) : l'API `IntersectionObserver` fait apparaître les blocs en fondu quand ils entrent à l'écran. Un délai en paramètre (`v-reveal="100"`) échelonne les éléments d'une liste. Les éléments déjà visibles au chargement ne sont pas animés.
- **Sélecteur de personnages** (`pages/personnages.vue`) : état réactif (`ref`, `computed`), navigation circulaire avec les flèches, synchronisation avec l'ancre de l'URL (`history.replaceState` et l'évènement `hashchange`).
- **Navigation précédent / suivant**, calculée à partir de la route courante.

ANIMATIONS CSS

- Entrée en cascade du texte (`.rise`, délai réglable par la variable `--rise-delay`).
- Lueur pulsée du nombre « 33 » sur le monolithe.
- Chute des pétales.
- Fondu entre les pages.

Toutes les animations sont désactivées quand l'utilisateur a demandé la réduction des animations dans son système (`prefers-reduced-motion`).

ACCESSIBILITÉ

Langue `fr` déclarée, lien d'évitement, `aria-label` sur les deux menus et sur le logo, textes alternatifs descriptifs sur les images, zones cliquables d'au moins 48 px, décor SVG masqué aux lecteurs d'écran (`aria-hidden`).

5. Logiciels, bibliothèques et ressources

Le site repose sur Nuxt et Nuxt UI. Les autres dépendances servent aux polices, aux icônes et à la vérification du code.

OUTIL	RÔLE	JUSTIFICATION
Nuxt 4	Framework Vue	Routage automatique à partir du dossier `pages/`, import automatique des composants, prérendu statique
Vue 3 (inclus avec Nuxt)	Interface réactive	Composants réutilisables et état réactif pour le sélecteur de personnages
Nuxt UI 4	Bibliothèque de composants	Fournit le template de départ et des composants accessibles (en-tête avec menu mobile, hero, cartes, fil d'Ariane), personnalisables sans être réécrits
Tailwind CSS 4 (inclus avec Nuxt UI)	Styles	Classes utilitaires et déclaration des tokens de couleur dans `@theme`
@nuxtjs/google-fonts	Polices	Charge Cormorant Garamond et Jost, proches de la typographie du jeu
@iconify-json/lucide	Icônes	Icônes de l'interface : flèches, information, menu
@iconify-json/simple-icons	Icônes	Collection d'icônes de marques détectée par Nuxt Icon
TypeScript	Typage	Détecte les erreurs avant l'exécution, par exemple dans la structure des données des personnages
ESLint (`@nuxt/eslint`)	Qualité du code	Style de code uniforme, vérifié automatiquement
vue-tsc	Vérification des types	Contrôle les types dans les fichiers `.vue`
Node.js 24 / npm 11	Environnement	Exécution des outils et gestion des dépendances
Git / GitHub	Versionnement	Historique du projet
GitHub Actions	Intégration continue	Lance `npm ci`, le lint et le typecheck à chaque push

Ressources graphiques : visuels officiels du jeu, © Sandfall Interactive / Kepler Interactive, crédités sous chaque image.

6. Sources de documentation

Documentation technique

- Nuxt — nuxt.com/docs
- Nuxt UI — ui.nuxt.com/docs
- Vue.js — vuejs.org/guide
- Tailwind CSS v4 — tailwindcss.com/docs
- Module Google Fonts pour Nuxt — google-fonts.nuxtjs.org
- MDN, API IntersectionObserver — developer.mozilla.org
- MDN, prefers-reduced-motion — developer.mozilla.org
- MDN, History API — developer.mozilla.org

Contenu du site

- Site officiel du jeu — expedition33.com
- Distribution des voix anglaises — expedition33.com
- TechRadar, palmarès des Game Awards 2025 — techradar.com

7. Intelligences artificielles utilisées

Claude (Anthropic) a servi d'assistant pour une partie du code, la relecture, la CI et la documentation. Les modifications de code ont été livrées sur des branches séparées, relues puis fusionnées dans `master`, et chaque version a été contrôlée par le lint, le typecheck et le build.

TÂCHE	CE QUE L'IA A FAIT	CONTRÔLE ET ADAPTATION
Animations	Transitions entre les pages, entrée en cascade du hero et des en-têtes, apparition au défilement, effets au survol des tuiles, fondu de la fiche personnage, désactivation avec `prefers-reduced-motion`	Code relu et testé sur le site en local avant d'être conservé
Images de la page Univers	Recherche de sources pour les visuels des figures du continent, puis intégration dans les cartes avec un texte alternatif, un crédit et l'option `fit="contain"`	Images choisies et ajoutées à la main dans `public/img/`. Branche `images-figures-continent` relue avant fusion
Favicon	Dessin du monolithe doré fissuré en SVG et déclinaison en ICO (16, 32 et 48 px)	Rendu vérifié dans l'onglet du navigateur, déclaration des deux formats ajustée dans `app.vue`
Passage de pnpm à npm	Retrait de pnpm du `package.json`, de la CI et du README, suppression de `pnpm-lock.yaml` et `pnpm-workspace.yaml`	Branche `passage-a-npm` relue avant fusion, installation vérifiée avec `npm ci`
Relecture avant le rendu	Vérification du projet par rapport à la consigne, repérage des dépendances inutilisées (Tailwind v3, packs d'icônes, i18n)	Chaque point vérifié dans le code. Suppressions faites à la main, puis contrôlées avec `npm run lint`, `npm run typecheck` et `npm run build`
Intégration continue	Diagnostic de l'échec de `npm ci` en CI : lockfile généré avec npm 11, CI sur Node 22 (npm 10)	Passage de la CI sur Node 24, puis vérification que le workflow GitHub Actions passe
Documentation	Premier jet du README et de ce document à partir du code du dépôt	Relu et corrigé pour correspondre au projet : chaque fichier, composant et dépendance cité vérifié dans le dépôt

Site non officiel, sans lien avec Sandfall Interactive ni Kepler Interactive. Clair Obscur : Expedition 33 et ses visuels appartiennent à leurs ayants droit.